e92260c7-1e24-4219-bee5-b8c16abff06d

<prev_user>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'ydb-platform/ydb' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ydb/ydb (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-10T12:46:06.335Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add insert rows functionality for fulltext indexed tables
</pr_title>

<pr_description>
Summary

Implemented insert rows functionality for fulltext indexed tables as requested.

Changes Made

1. Query Optimizer Layer (`ydb/core/kqp/opt/physical/effects/kqp_opt_phy_insert_index.cpp`)
- ✅ Added `MakeInsertFulltextIndexRows` function that builds a lambda to:
  - Read fulltext index settings from `TIndexDescription`
  - Extract text from input rows
  - Call `FulltextAnalyze` callable to tokenize text according to index settings
  - Generate index rows with structure: `(token, primary_key_columns..., data_columns...)`
- ✅ Integrated into `KqpBuildInsertIndexStages` to handle `GlobalFulltext` index type
- ✅ Properly handles both covered and non-covered index columns

2. MKQL Compiler Layer (`ydb/core/kqp/query_compiler/kqp_mkql_compiler.cpp`)
- ✅ Added `FulltextAnalyze` callable that calls `TKqpProgramBuilder::FulltextAnalyze`
- ✅ Properly passes text and serialized analyzer settings to runtime layer

3. Runtime Layer (`ydb/core/kqp/runtime/`)
- ✅ Implemented `TKqpProgramBuilder::FulltextAnalyze` method in `kqp_program_builder.{cpp,h}`
- ✅ Creates `FulltextTokenize` callable using `TCallableBuilder`
- ✅ Validates input types (text and settings must be strings)
- ✅ Returns `List<String>` type
- ✅ Created `kqp_fulltext_tokenize.cpp/h` with `TFulltextTokenizeWrapper` computation node
- ✅ Deserializes `Ydb::Table::FulltextIndexSettings::Analyzers` from settings parameter
- ✅ Calls `NKikimr::NFulltext::Analyze` function (same as used by `TBuildFulltextIndexScan`)
- ✅ Returns list of tokens as `TUnboxedValue`
- ✅ Registered in `GetKqpBaseComputeFactory` in `kqp_compute.cpp`
- ✅ Added to `ya.make` build configuration

4. Tests (`ydb/core/kqp/ut/indexes/kqp_indexes_fulltext_ut.cpp`)
- ✅ Added `InsertRow` test - verifies insert into table with basic fulltext index
- ✅ Added `InsertRowCovered` test - verifies insert into table with covered fulltext index
- ✅ Tests verify correct tokenization and index row generation

Implementation Details

The implementation follows the same pattern as other KQP program builder methods:
- `TKqpProgramBuilder::FulltextAnalyze` creates a callable using `TCallableBuilder`
- Uses `FlatMap` to generate multiple index rows from a single input row (one per token)
- Properly handles serialization of analyzer settings for runtime execution
- Reuses existing `NKikimr::NFulltext::Analyze` function for consistency with index building

Checklist
- [x] Analyze existing code structure
- [x] Implement MakeInsertFulltextIndexRows function
- [x] Integrate into KqpBuildInsertIndexStages
- [x] Implement TKqpProgramBuilder::FulltextAnalyze method
- [x] Implement FulltextAnalyze callable
- [x] Implement FulltextTokenize MKQL computation node
- [x] Add test cases
- [ ] Build and test (requires full build environment)

</pr_description>

Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@kunga</author>
@copilot there is no `NewCallable` method

I think you should implement TKqpProgramBuilder::FulltextAnalyze
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot there is no `NewCallable` method
>
> I think you should implement
TKqpProgramBuilder::FulltextAnalyze

Fixed in commit bd6d993. Implemented
`TKqpProgramBuilder::FulltextAnalyze` method following the same
pattern as `KqpEnsure` and `KqpIndexLookupJoin`. The method
validates input types and creates a `FulltextTokenize` callable
using `TCallableBuilder`.
</comment_old>

<comment_new>
<comment_id>3389894408</comment_id>
<author>@kunga</author>
@copilot

```
  </comment_new>
```

</pr_comments>

</comments>

The last **8** git commits in this branch are the changes you
have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the
<comments> before making any changes. You will only address the
<comment_new> comments. Reply to new comments as you address them.
Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a
suggestion.
    - Is it addressed to you or someone else, e.g. "Hey
@username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can
stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and
ask for clarification.
* If an action or explanation is needed, follow the steps below.

</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.

<prev_assistant>

<empty>

<current>

<request>

You have been given comments on the previous commits you made in the repository.

You are working on an issue in the 'ydb-platform/ydb' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/ydb/ydb (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2025-10-10T12:49:24.270Z</current_datetime>

Consider the following problem statement:

<problem_statement>

This section includes details on the pull request with code changes you have already submitted to fix the problem above.

<pr_title>
Add insert rows functionality for fulltext indexed tables
</pr_title>

<pr_description>

```
## Summary

Implemented insert rows functionality for fulltext indexed
tables as requested.

## Changes Made

### 1. Query Optimizer Layer (`ydb/core/kqp/opt/physical/
effects/kqp_opt_phy_insert_index.cpp`)
- ✅ Added `MakeInsertFulltextIndexRows` function that builds a
lambda to:
  - Read fulltext index settings from `TIndexDescription`
  - Extract text from input rows
  - Call `FulltextAnalyze` callable to tokenize text according
to index settings
  - Generate index rows with structure: `(token,
primary_key_columns..., data_columns...)`
- ✅ Integrated into `KqpBuildInsertIndexStages` to handle
`GlobalFulltext` index type
- ✅ Properly handles both covered and non-covered index columns

### 2. MKQL Compiler Layer (`ydb/core/kqp/query_compiler/
kqp_mkql_compiler.cpp`)
- ✅ Added `FulltextAnalyze` callable that calls
`TKqpProgramBuilder::FulltextAnalyze`
- ✅ Properly passes text and serialized analyzer settings to
runtime layer

### 3. Runtime Layer (`ydb/core/kqp/runtime/`)
- ✅ Implemented `TKqpProgramBuilder::FulltextAnalyze` method in
`kqp_program_builder.{cpp,h}`
- ✅ Creates `FulltextTokenize` callable using
`TCallableBuilder`
- ✅ Validates input types (text and settings must be strings)
- ✅ Returns `List<String>` type
- ✅ Created `kqp_fulltext_tokenize.cpp/h` with
`TFulltextTokenizeWrapper` computation node
- ✅ Deserializes `Ydb::Table::FulltextIndexSettings::Analyzers`
from settings parameter
- ✅ Calls `NKikimr::NFulltext::Analyze` function (same as used
by `TBuildFulltextIndexScan`)
- ✅ Returns list of tokens as `TUnboxedValue`
- ✅ Registered in `GetKqpBaseComputeFactory` in
`kqp_compute.cpp`
- ✅ Added to `ya.make` build configuration

### 4. Tests (`ydb/core/kqp/ut/indexes/
```

```
kqp_indexes_fulltext_ut.cpp`)
- ✅ Added `InsertRow` test - verifies insert into table with
basic fulltext index
- ✅ Added `InsertRowCovered` test - verifies insert into table
with covered fulltext index
- ✅ Tests verify correct tokenization and index row generation

## Implementation Details

The implementation follows the same pattern as other KQP program
builder methods:
- `TKqpProgramBuilder::FulltextAnalyze` creates a callable using
`TCallableBuilder`
- Uses `FlatMap` to generate multiple index rows from a single
input row (one per token)
- Properly handles serialization of analyzer settings for
runtime execution
- Reuses existing `NKikimr::NFulltext::Analyze` function for
consistency with index building

## Checklist
- [x] Analyze existing code structure
- [x] Implement MakeInsertFulltextIndexRows function
- [x] Integrate into KqpBuildInsertIndexStages
- [x] Implement TKqpProgramBuilder::FulltextAnalyze method
- [x] Implement FulltextAnalyze callable
- [x] Implement FulltextTokenize MKQL computation node
- [x] Add test cases
- [ ] Build and test (requires full build environment)









</pr_description>

## Comments on the PR (you are @copilot in this section)

<comments>

<pr_comments>

<comment_old>
<author>@kunga</author>
```

@copilot there is no `NewCallable` method

I think you should implement TKqpProgramBuilder::FulltextAnalyze
</comment_old>

<comment_old>
<author>@copilot</author>

> @copilot there is no `NewCallable` method
>
> I think you should implement
TKqpProgramBuilder::FulltextAnalyze

Fixed in commit bd6d993. Implemented
`TKqpProgramBuilder::FulltextAnalyze` method following the same
pattern as `KqpEnsure` and `KqpIndexLookupJoin`. The method
validates input types and creates a `FulltextTokenize` callable
using `TCallableBuilder`.
</comment_old>

<comment_old>
<author>@kunga</author>
@copilot

```
  </comment_old>

<comment_old>
<author>@copilot</author>

> @copilot
>
> ```
...

The comment appears to be incomplete. Could you please provide
the full feedback or question?
</comment_old>

<comment_new>
<comment_id>3389907664</comment_id>
<author>@kunga</author>
@copilot

</comment_new>

</pr_comments>
```

</comments>

The last **8** git commits in this branch are the changes you have made so far. Use those as your change commit history.
</problem_statement>

Analyze and determine if action or explanation is needed
<rules_for_making_changes>
* Start by fully understanding <problem_statement> and the <comments> before making any changes. You will only address the <comment_new> comments. Reply to new comments as you address them. Ignore comments that are not actionable or directed at you.
* For each new comment, determine:
    - If it a request for changes, a question on the code or a suggestion.
    - Is it addressed to you or someone else, e.g. "Hey @username, ..." or "@username can you ...".
    - Is it a general comment or praise.
* If there are no comments you need to address or reply to, you can stop now without exploring or explaining yourself.
* If the user's ask is not clear, you can reply to the comment and ask for clarification.
* If an action or explanation is needed, follow the steps below.
</rules_for_making_changes>

Ensure that you lint, build and test early and iteratively to validate each code change thoroughly.